TECH REFERENCE

# PLE + adaTT
# 기술 참조서

Progressive Layered Extraction · Adaptive Task Transfer
CGC Gate · GroupTaskExpertBasket · Logit Transfer · 2-Phase Training

프로젝트
**AWS PLE for Financial**
PLE-Cluster-adaTT Architecture

버전
**v1.0**
2026-04-01

> 이 문서는 PLE-Cluster-adaTT 아키텍처의 핵심 구조를 기술한다. Shared-Bottom에서 PLE까지의 발전, 이종 전문가 Basket 설계, adaTT gradient 기반 태스크 전이, 태스크 그룹 구조, 학습 전략, 구현 시 FP16/GradScaler 주의사항을 포함한다.

# 목 차

1. PLE 아키텍처 . . . . . 4
1.1 Multi-Task Learning의 동기 . . . . . 4
1.2 Shared-Bottom에서 PLE까지의 발전 . . . . . 4
Shared-Bottom (Caruana, 1997) . . . . . 4
MMoE (Ma et al., KDD 2018) . . . . . 4
PLE (Tang et al., RecSys 2020) . . . . . 4
1.3 CGC (Customized Gate Control) . . . . . 5
CGCLayer (가중합 방식) . . . . . 5
CGCAttention (블록 스케일링 방식) . . . . . 5
초기 Bias 설정 (domain_experts) . . . . . 6
Entropy 정규화 . . . . . 6
차원 정규화 (v3.3) . . . . . 6
2. 이종 전문가 Basket . . . . . 7
2.1 Pool/Basket 패턴 . . . . . 7
2.2 Expert 선정 기준 . . . . . 7
2.3 Expert 라우팅 . . . . . 7
3. adaTT — Adaptive Task-aware Transfer . . . . . 9
3.1 Motivation: Negative Transfer . . . . . 9
3.2 Gradient Cosine Similarity . . . . . 9
3.3 EMA 안정화 . . . . . 9
3.4 Transfer-Enhanced Loss . . . . . 10
3.5 Transfer Weight 계산 (4-stage pipeline) . . . . . 10
3.6 3-Phase Schedule . . . . . 11
3.7 Group Prior . . . . . 11
3.8 Negative Transfer 감지 및 차단 . . . . . 11
3.9 Attention 메커니즘과의 유비 . . . . . 11
4. 태스크 그룹 . . . . . 13
4.1 4개 Financial DNA 그룹 . . . . . 13
4.2 Intra/Inter Strength 설계 . . . . . 13
4.3 Logit Transfer 3방식 . . . . . 13
4.4 GroupTaskExpertBasket (v3.2) . . . . . 14
Soft Routing . . . . . 14
5. 학습 전략 . . . . . 15
5.1 2-Phase Training . . . . . 15
Phase 1: Shared Expert Pretrain . . . . . 15
Phase 2: Cluster Finetune . . . . . 15
5.2 Uncertainty Weighting . . . . . 15
5.3 Focal Loss . . . . . 16
5.4 AMP (Automatic Mixed Precision) . . . . . 16
6. 구현 참고사항 . . . . . 17
6.1 FP16 주의점 . . . . . 17
Focal Loss float32 캐스팅 . . . . . 17

adaTT Gradient 추출과 autocast ..... 17
6.2 GradScaler Guard ..... 17
Gradient 추출 빈도 최적화 ..... 17
torch.compiler.disable ..... 17
6.3 Phase 2 Frozen Layer ..... 17
CGC-adaTT 동기화 ..... 18
6.4 Loss 계산 파이프라인 전체 순서 ..... 18
6.5 디버깅 가이드 ..... 18
References ..... 20

> **설계 vs 구현 참고**
>
> 본 문서는 풀뱅크 설계(734D)를 기준으로 작성되었습니다. 현재 Santander 벤치마크 구현은 316D (12 feature groups)입니다.

# 1. PLE 아키텍처

## 1.1 Multi-Task Learning의 동기

AIOps 추천 시스템은 18개 태스크를 동시 예측해야 한다. 태스크 간 공유되는 패턴을 활용하면 데이터 효율이 극적으로 향상된다. 총 손실은 가중합으로 정의된다:

$$\mathcal{L}_{\text{MTL}} = \sum_{k=1}^{K} w_k \cdot \mathcal{L}_k\Big(f_k\Big(\boldsymbol{h}_{\text{shared}(\boldsymbol{x})}\Big)\Big)$$

$\boldsymbol{h}_{\text{shared}}$는 어느 한 태스크에만 과적합하기 어렵다. 모든 태스크에 동시에 유용한 표현만이 살아남으며, 이것이 **태스크 간 상호 정규화(inter-task regularization)**이다.

## 1.2 Shared-Bottom에서 PLE까지의 발전

**Shared-Bottom (Caruana, 1997)**

모든 태스크가 하나의 trunk을 공유한 뒤 태스크별 head로 분기한다.

$$\boldsymbol{h} = f_{\text{shared}(\boldsymbol{x})} \quad \rightarrow \quad \hat{y}_k = f_k^{\text{tower}(\boldsymbol{h})}$$

구현이 단순하고 파라미터 효율적이나, 태스크 간 관련성이 낮을 때 **Negative Transfer**가 심각하다.

**MMoE (Ma et al., KDD 2018)**

$N$개의 동일 구조 Expert를 두고 태스크별 gate가 가중합을 결정한다.

$$\boldsymbol{h}_k = \sum_{i=1}^{N} g_{k,i} \cdot f_i^{\text{expert}(\boldsymbol{x})}, \quad \boldsymbol{g}_k = \text{Softmax}\Big(\boldsymbol{W}_k^{\text{gate}} \cdot \boldsymbol{x}\Big)$$

태스크별로 다른 Expert 조합이 가능하나, 모든 gate가 동일 Expert를 선택하는 **Expert Collapse** 문제가 발생한다.

**PLE (Tang et al., RecSys 2020)**

Expert를 **Shared Expert** $\mathcal{E}^s$와 **Task-specific Expert** $\mathcal{E}^k$로 명시 분리하고, CGC gate가 두 종류의 Expert를 최적 비율로 결합한다.

$$\boldsymbol{h}_k = \sum_{i=1}^{|\mathcal{E}^s|} g_{k,i}^s \cdot \boldsymbol{e}_i^s + \sum_{j=1}^{|\mathcal{E}^k|} g_{k,j}^k \cdot \boldsymbol{e}_j^k$$

$\boldsymbol{e}_i^s$: $i$번째 Shared Expert 출력, $\boldsymbol{e}_j^k$: 태스크 $k$의 $j$번째 Task Expert 출력
$g_{k,i}^s, g_{k,j}^k$: CGC gate 가중치 (Softmax 정규화)

PLE가 해결하는 문제:

1. **Negative Transfer 완화**: Task-specific Expert가 간섭 없이 특화 패턴 학습

2. **Expert Collapse 방지**: Shared/Task Expert 역할이 자연스럽게 분리
3. **Progressive Extraction**: 여러 layer를 쌓아 저수준에서 고수준으로 정보 정제

| 구분 | Shared-Bottom | MMoE | PLE |
|---|---|---|---|
| Expert 구조 | 단일 Shared trunk | N개 Expert 전체 공유 | Shared + Task-specific 분리 |
| 게이팅 | 없음 | 태스크별 Softmax gate | CGC: Shared + Task Expert 결합 |
| Negative Transfer | 높음 | 중간 (Expert Collapse) | 낮음 (명시적 분리) |
| Expert Collapse | N/A | 높음 | 낮음 |

## 1.3 CGC (Customized Gate Control)

본 구현에서는 두 가지 CGC 변형이 존재하며, 역할과 출력 형태가 다르다.

### CGCLayer (가중합 방식)

원본 PLE 논문의 CGC로, 태스크별 gate가 Shared + Task Expert 출력의 **가중합**을 산출한다. 출력 차원은 `expert_hidden_dim` 으로 고정된다 (Expert 수와 무관).

$$\boldsymbol{h}_k = \sum_{i=1}^{N} g_{k,i} \cdot \boldsymbol{e}_i, \quad \boldsymbol{g}_k = \mathrm{Softmax}\left(\boldsymbol{W}_k^{\mathrm{gate}} \cdot \boldsymbol{x}\right) \in \mathbb{R}^N$$

$N$: Shared + Task Expert 총 수
$\boldsymbol{e}_i \in \mathbb{R}^{d_{\text{expert}}}$: $i$번째 Expert 출력
결과: $\boldsymbol{h}_k \in \mathbb{R}^{d_{\text{expert}}}$ — Expert 수에 무관한 고정 차원

### CGCAttention (블록 스케일링 방식)

이종 Expert의 출력을 **연결(concatenation)**한 뒤 태스크별 attention weight로 각 Expert 블록을 스케일링한다. 출력 차원은 모든 Expert 출력 차원의 합이다.

$$\boldsymbol{w}_k = \mathrm{Softmax}(\boldsymbol{W}_k \cdot \boldsymbol{h}_{\text{shared}} + \boldsymbol{b}_k) \in \mathbb{R}^8$$

$$\tilde{\boldsymbol{h}}_{k,i} = w_{k,i} \cdot \boldsymbol{h}_i^{\text{expert}} \quad \text{for } i = 1, \ldots, 8$$

$$\boldsymbol{h}_k^{\text{cgc}} = \left[\tilde{\boldsymbol{h}}_{k,1} \parallel \tilde{\boldsymbol{h}}_{k,2} \parallel \ldots \parallel \tilde{\boldsymbol{h}}_{k,8}\right] \in \mathbb{R}^{576}$$

$\boldsymbol{W}_k \in \mathbb{R}^{8\times 576}$: 태스크 $k$의 gate weight
$w_{k,i}$: 태스크 $k$가 Expert $i$에 부여하는 attention weight
결과: 동일 576D이지만 태스크마다 Expert별 기여 비중이 다름

본 PLE-Cluster-adaTT 구현에서는 8개 이종 Shared Expert에 대해 **CGCAttention** (블록 스케일링)을 사용한다. Transformer Attention과 유사하게 **Query**(gate weight), **Key**(shared representation), **Value**(Expert output)의 관계로 "관련성에 비례하여 정보를 선택적으로 결합"한다.

### 초기 Bias 설정 (domain_experts)

태스크별 `domain_experts` config로 선호 Expert에 `bias_high=1.0`, 비선호에 `bias_low=-1.0`을 부여하여 학습 초기 Expert 선택을 유도한다.

### Entropy 정규화

Expert Collapse 방지를 위해 gate 분포의 엔트로피를 정규화한다:

$$\mathcal{L}_{\text{entropy}} = \lambda_{\text{ent}} \cdot \left(-\frac{1}{|\mathcal{T}|}\right) \sum_{k \in \mathcal{T}} H(\boldsymbol{w}_k), \quad H(\boldsymbol{w}_k) = -\sum_{i=1}^{8} w_{k,i} \cdot \log(w_{k,i})$$

$\lambda_{\text{ent}} = 0.01$: 음의 엔트로피를 최소화하면 엔트로피가 증가하여 Expert 활용 분산 유도

### 차원 정규화 (v3.3)

이종 Expert 차원 비대칭(128D vs 64D)에 의한 기여도 불균형을 보정한다:

$$\text{scale}_i = \sqrt{\frac{\text{mean_dim}}{\text{dim}_i}}, \quad \text{mean_dim} = \frac{128 + 64 \times 7}{8} = 72.0$$

unified_hgcn (128D): scale ≈ 0.750 (감쇠), 나머지 (64D): scale ≈ 1.061 (증폭)

# 2. 이종 전문가 Basket

## 2.1 Pool/Basket 패턴

본 시스템은 PLE의 동일 구조 Expert 대신 **8개 이종 도메인 Expert**를 Shared Expert Pool로 결합한다. 각 Expert는 입력 데이터를 전혀 다른 수학적 관점으로 해석하며, CGC Gate가 태스크별 최적 조합을 학습한다.

| Expert | 입력 | 출력 | 역할 및 대체 불가능성 |
|---|---|---|---|
| DeepFM | 정규화 644D | 64D | FM의 $O(nk)$ 2차 교차를 명시적으로 포착 |
| LightGCN | 사전 계산 64D | 64D | 이분 그래프 기반 "비슷한 고객" 협업 신호 |
| Unified HGCN | 47D | 128D | 쌍곡 공간에서 MCC 계층 구조 인코딩 |
| Temporal | 시퀀스 입력 | 64D | Mamba+LNN+Transformer 시간 패턴 앙상블 |
| PersLay | Persistence Diagram | 64D | 소비 패턴의 위상적 구조(루프, 클러스터) |
| Causal | 정규화 644D | 64D | SCM/NOTEARS 기반 방향성 인과 관계 추출 |
| Optimal Transport | 정규화 644D | 64D | Sinkhorn Wasserstein 분포 기하학 |
| RawScale | 원시 90D | 64D | 정규화 전 멱법칙 분포 정보 보존 |

결합 차원은 $7 \times 64 + 1 \times 128 = 576$D이다.

$$\boldsymbol{h}_{\text{shared}} = [\text{unified_hgcn}_{128\text{D}} \parallel \text{perslay}_{64\text{D}} \parallel \text{deepfm}_{64\text{D}} \parallel \text{temporal}_{64\text{D}} \parallel \text{lightgcn}_{64\text{D}} \parallel \text{causal}_{64\text{D}} \parallel \text{OT}_{64\text{D}} \parallel \text{raw_scale}_{64\text{D}}]$$

## 2.2 Expert 선정 기준

8개 Expert는 동일 고객 데이터의 **근본적으로 다른 수학 구조**를 추출한다:

- **DeepFM/Causal/OT**: 동일 정규화 644D를 입력받지만, 대칭 교차(FM), 비대칭 인과(DAG), 분포 거리(Wasserstein)라는 다른 구조를 포착
- **Temporal**: 시퀀스 데이터의 시간적 동역학 ($[B, 180, 16]$, $[B, 90, 8]$)
- **PersLay**: 위상적 불변량 (Betti number, persistence)
- **Unified HGCN**: 쌍곡 기하학에서의 계층 관계
- **LightGCN**: 고객-가맹점 그래프의 협업 필터링 신호
- **RawScale**: 정규화 시 손실되는 원시 스케일/멱법칙 분포 패턴

## 2.3 Expert 라우팅

각 Expert는 config의 `shared_experts` 섹션에서 활성화되며, `_forward_shared_experts()` 에서 Expert 이름에 따라 서로 다른 입력이 디스패치된다. 입력 데이터가 `None` 인 경우 **zero tensor fallback**을 수행하며, CGC 게이팅이 해당 Expert의 가중치를 자동으로 낮춘다.

HMM Triple-Mode는 별도 입력 경로로 처리된다:

| HMM 모드 | 입력 | 시간 스케일 | 대상 태스크 |
|---|---|---|---|
| Journey | 16D | daily | has_nba, engagement_score, cross_sell_count, will_acquire_* |
| Lifecycle | 16D | monthly | churn_signal, product_stability, tenure_stage, segment_prediction |
| Behavior | 16D | monthly | income_tier, spend_level, nba_primary, next_mcc, ... |

각 모드는 10D base 상태 확률 + 6D ODE dynamics로 구성되며, 16D를 32D로 프로젝션하여 CGC 출력과 concat 된다:

$$\boldsymbol{h}_{\text{hmm}}^{m} = \text{SiLU}(\text{LayerNorm}(\text{Linear}_{16\to 32}(\boldsymbol{x}_{\text{hmm}}^{m})))$$

# 3. adaTT — Adaptive Task-aware Transfer

## 3.1 Motivation: Negative Transfer

18개 태스크가 Shared Expert 파라미터를 공유할 때, gradient 충돌로 인한 Negative Transfer가 발생한다. 고정 타워 MTL의 세 가지 한계:

1. **일방적 공유**: 태스크 간 간섭을 감지/조절할 메커니즘 부재
2. **상호작용 미측정**: 어떤 태스크 쌍이 돕고 해치는지 정량화 불가
3. **시간 변화 미반영**: 학습 단계별 태스크 관계 변화를 추적 불가

adaTT는 이를 각각 **선택적 전이**, **gradient cosine similarity**, **3-Phase schedule**로 해결한다.

## 3.2 Gradient Cosine Similarity

두 태스크 $i$, $j$의 Shared Expert 파라미터 $\theta$에 대한 gradient를 $\boldsymbol{g}_i = \nabla_\theta \mathcal{L}_i$, $\boldsymbol{g}_j = \nabla_\theta \mathcal{L}_j$라 하면:

$$\cos(\theta_{i,j}) = \frac{\boldsymbol{g}_i \cdot \boldsymbol{g}_j}{\|\boldsymbol{g}_i\| \cdot \|\boldsymbol{g}_j\|}$$

$\boldsymbol{g}_i \in \mathbb{R}^d$: 태스크 $i$의 flattened gradient 벡터
결과 범위: $[-1, 1]$ — 양수 = positive transfer, 음수 = negative transfer

**코사인 유사도를 사용하는 이유**:
1. **스케일 불변성**: 태스크별 loss 규모 차이에 영향받지 않고 순수 방향 비교
2. **해석 용이**: $[-1, 1]$ 범위로 정규화되어 직관적 해석 가능
3. **효율적 계산**: gradient 행렬 $\boldsymbol{G} \in \mathbb{R}^{n \times d}$를 L2 정규화 후 $\hat{\boldsymbol{G}}\hat{\boldsymbol{G}}^\top$으로 모든 $n^2$ 유사도를 단일 GEMM으로 계산

```
norms = grad_matrix.norm(dim=1, keepdim=True).clamp(min=1e-8)
normalized = grad_matrix / norms
affinity = torch.mm(normalized, normalized.t())  # [n_tasks, n_tasks]
```

## 3.3 EMA 안정화

단일 배치의 gradient는 노이즈가 크므로 EMA로 평활화한다:

$$\boldsymbol{A}_t = \alpha \cdot \boldsymbol{A}_{t-1} + (1 - \alpha) \cdot \cos(\theta_t)$$

$\alpha = 0.9$: effective window $\approx \frac{1}{1-\alpha} = 10$ 관측
신호처리 관점: IIR 1차 저역통과 필터 $H(z) = \frac{1-\alpha}{1-\alpha z^{-1}}$

EMA 업데이트 후 반드시 `.clamp(-1.0, 1.0)` 적용 — 부동소수점 오차 누적으로 코사인 유사도가 $[-1, 1]$ 범위를 벗어나면 후속 연산에서 NaN 발생 가능.

## 3.4 Transfer-Enhanced Loss

각 태스크 $i$에 대한 adaTT 강화 손실:

$$\mathcal{L}_i^{\text{adaTT}} = \mathcal{L}_i + \lambda \cdot \sum_{j \neq i} w_{i \to j} \cdot \mathcal{L}_j$$

$\mathcal{L}_i$: 태스크 $i$의 원본 손실
$\lambda = 0.1$: 다른 태스크 의견의 반영 비율 (10%)
$w_{i \to j}$: 태스크 $i$에 대한 태스크 $j$의 전이 가중치 (softmax 정규화)

본 시스템은 softmax 외에 sigmoid gate도 지원한다 (NeurIPS 2024, Nguyen et al.). Sigmoid gate는 expert 간 독립적 기여를 허용하여, 이종 expert 환경에서의 불필요한 경쟁을 제거한다. 설정: `gate_type: "sigmoid"` (pipeline.yaml 또는 HP).

Gradient에 대한 영향:

$$\nabla_\theta \mathcal{L}_i^{\text{adaTT}} = \nabla_\theta \mathcal{L}_i + \lambda \sum_{j \neq i} w_{i \to j} \nabla_\theta \mathcal{L}_j$$

두 번째 항은 공유 파라미터를 여러 태스크에 유리한 방향으로 조향하는 **보정 벡터**이다.

## 3.5 Transfer Weight 계산 (4-stage pipeline)

$$\begin{aligned} \boldsymbol{R} &= (\boldsymbol{W} + \boldsymbol{A}) \cdot (1 - r) + \boldsymbol{P} \cdot r \\ \boldsymbol{R}_{i,j} &\leftarrow 0 \quad \text{if } \boldsymbol{A}_{i,j} < \tau_{\text{neg}} \\ \boldsymbol{R}_{i,i} &= 0 \\ w_{i \to j} &= \text{softmax}\left(\frac{\boldsymbol{R}_{i,j}}{T}\right) \end{aligned}$$

$\boldsymbol{W}$: 학습 가능한 전이 가중치 ( `nn.Parameter` , 초기값 0)
$\boldsymbol{A}$: EMA 친화도 행렬
$\boldsymbol{P}$: Group Prior 행렬 (도메인 지식)
$r$: Prior blend ratio (0.5 $\to$ 0.1로 annealing)
$\tau_{\text{neg}} = -0.1$: Negative transfer 차단 임계값
$T = 1.0$: Softmax temperature

**각 단계의 의미**:
1. 학습 가능 가중치 $\boldsymbol{W}$와 관측 친화도 $\boldsymbol{A}$를 합산, 도메인 Prior $\boldsymbol{P}$와 혼합
2. 친화도가 임계값 이하인 경로를 0으로 차단 (negative transfer blocking)
3. 자기 전이 제외 (대각선 0)
4. Softmax로 확률 분포 생성 — 합이 1이므로 태스크 수에 무관하게 스케일 일정

`max_transfer_ratio = 0.5` : Transfer loss가 원본 loss의 50%를 초과할 수 없다.

## 3.6 3-Phase Schedule

| Phase | 기간 | 동작 | 목적 |
|---|---|---|---|
| 1: Warmup | epoch 0 `warmup_epochs` | 친화도 계산만, transfer loss 미적용 | 안정적 친화도 데이터 축적 |
| 2: Dynamic | `warmup_epochs` `freeze_epoch` | 전이 활성, Prior blend annealing | 태스크 관계 학습 및 적용 |
| 3: Frozen | `freeze_epoch` 종료 | 전이 가중치 고정 ( `detach` ) | Fine-tuning 안정화, gradient overhead 제거 |

> **검증: freeze_epoch > warmup_epochs**
>
> Phase 2가 완전히 스킵되면 학습된 친화도가 전이에 반영되지 않아 adaTT 사용의 의미가 없어진다. 초기화 시 `ValueError` 로 조기 차단.

## 3.7 Group Prior

Prior blend ratio는 학습 진행에 따라 선형 감소한다:

$$r(e) = r_{\text{start}} - (r_{\text{start}} - r_{\text{end}}) \cdot \min\left(\frac{e - e_{\text{warmup}}}{e_{\text{freeze}} - e_{\text{warmup}}}, 1.0\right)$$

$r_{\text{start}} = 0.5 \rightarrow r_{\text{end}} = 0.1$: 학습 초기 도메인 지식 의존, 후반 관측 친화도 신뢰

Bayesian 해석: Prior($\boldsymbol{P}$)에서 Posterior(관측 기반 $\boldsymbol{A}$)로의 전환. 데이터가 부족한 초기에는 prior에 의존하고, 데이터가 축적되면 likelihood를 따른다.

## 3.8 Negative Transfer 감지 및 차단

$$\boldsymbol{R}_{i,j} = \begin{cases} \boldsymbol{R}_{i,j} & \text{if } \boldsymbol{A}_{i,j} > \tau_{\text{neg}} \\ 0 & \text{otherwise} \end{cases}$$

임계값 $\tau_{\text{neg}} = -0.1$ (0이 아닌 이유): 약한 음의 상관은 노이즈일 수 있으므로 허용하고, 명확한 반대 방향 gradient만 차단한다. 0으로 설정하면 과도한 경로 차단으로 adaTT 효과가 약화된다.

진단 API: `detect_negative_transfer()` 가 `{"churn_signal": ["has_nba", "engagement_score"]}` 형태로 negative transfer 쌍을 반환한다.

## 3.9 Attention 메커니즘과의 유비

adaTT는 **태스크 공간에서의 Attention**으로 해석할 수 있다:

| 역할 | Transformer Self-Attention | adaTT Task Transfer |
|---|---|---|
| Query | 현재 토큰의 질의 | 현재 태스크의 gradient 방향 |
| Key | 다른 토큰의 응답 가능성 | 다른 태스크의 gradient 방향 |

| | | |
|---|---|---|
| 유사도 | $\boldsymbol{Q}\frac{\boldsymbol{K}^\top}{\sqrt{d_k}}$ | gradient cosine similarity |
| 정규화 | softmax | softmax (temperature $T$) |
| Value | 다른 토큰의 정보 | 다른 태스크의 loss 값 |
| 출력 | 가중 합산된 context | transfer loss |

Hypernetwork(Ha et al., 2017)와 비교하면, adaTT는 학습된 태스크 임베딩 대신 **관측 gradient**를 조건 신호로 사용하여 지연 없이 태스크 관계 변화에 적응한다.

# 4. 태스크 그룹

## 4.1 4개 Financial DNA 그룹

도메인 지식에 기반하여 18개 태스크를 4개 그룹으로 분류한다:

| 그룹 | 멤버 | Intra | Inter | 비즈니스 의미 |
|---|---|---|---|---|
| Engagement | has_nba, engagement_score, cross_sell_count, will_acquire_deposits, will_acquire_investments, will_acquire_accounts, will_acquire_lending, will_acquire_payments | 0.8 | 0.3 | 고객 참여/전환 |
| Lifecycle | churn_signal, product_stability, tenure_stage, segment_prediction | 0.7 | 0.3 | 고객 생애주기 |
| Value | income_tier, spend_level, nba_primary | 0.6 | 0.3 | 고객 가치/행동 패턴 |
| Consumption | next_mcc, mcc_diversity_trend, top_mcc_shift | 0.7 | 0.3 | 소비 패턴 분석 |

**Intra-group strength**: 같은 그룹 내 태스크 간의 전이 강도. Engagement 그룹(0.8)이 가장 높다 — 상품 획득 태스크들은 유사 구조로 강한 positive transfer 기대.

**Inter-group strength**: 그룹 간 전이 강도. 0.3으로 보수적 — cross-group transfer는 gradient 관측으로 검증된 후에만 활성화.

## 4.2 Intra/Inter Strength 설계

Group Prior 행렬 $P$는 그룹 구조에서 자동 생성된다:

- 같은 그룹 내: intra_strength (0.6 0.8)
- 다른 그룹 간: inter_group_strength (0.3)
- 대각선: 0 (자기 전이 제외)
- 행 정규화로 합을 1로 맞춤

## 4.3 Logit Transfer 3방식

태스크 간 **명시적 정보 전달**을 위한 세 가지 전이 방식:

| Source | Target | 유형 | 강도 | 비즈니스 의미 |
|---|---|---|---|---|
| has_nba | nba_primary | Sequential | 0.5 | NBA 존재 → 주력 상품 결정 |
| churn_signal | product_stability | Inverse | 0.5 | 이탈 신호의 역 ≈ 상품 안정성 |

| | | | | |
|---|---|---|---|---|
| engagement_score | cross_sell_count | Feature | 0.5 | 참여도 → 교차판매 기회 |
| next_mcc | mcc_diversity_trend | Feature | 0.5 | 다음 소비처 → 다양성 추세 |
| spend_level | income_tier | Feature | 0.5 | 지출 수준 → 소득 구간 |

전이 메커니즘 (residual 형태):

$$\boldsymbol{h}^t_{\text{tower}} = \boldsymbol{h}^t_{\text{expert}} + \alpha \cdot \text{SiLU}(\text{LayerNorm}(\text{Linear}(\text{pred}^s)))$$

$\alpha = 0.5$: transfer strength. 프로젝션 가중치가 0에 수렴하면 자연스럽게 전이 무시.

실행 순서는 `task_relationships` config의 의존 관계를 **Kahn's algorithm**(위상 정렬)으로 자동 도출한다. 순환이 감지되면 fallback 순서로 전환된다.

## 4.4 GroupTaskExpertBasket (v3.2)

기존 클러스터×태스크 독립 MLP 대비 **88% 파라미터 감소** (∼3.0M → ∼362K). 같은 태스크 그룹은 GroupEncoder를 공유하고, ClusterEmbedding으로 차별화한다:

$$\boldsymbol{e}_{\text{cluster}} = \text{Embedding}(\text{cluster_id}) \in \mathbb{R}^{32}$$

$$\boldsymbol{x}_{\text{input}} = \left[\text{CGC_output}_{\text{576D}} \parallel \text{HMM_proj}_{\text{32D}} \parallel \boldsymbol{e}_{\text{cluster}_{\text{32D}}}\right] \in \mathbb{R}^{640}$$

$$\boldsymbol{h}_{\text{expert}} = \text{MLP}_{640\to128\to64\to32}(\boldsymbol{x}_{\text{input}})$$

**Soft Routing**

클러스터 경계 샘플에 대해 GMM 사후 확률로 임베딩을 혼합한다:

$$\boldsymbol{e}_{\text{cluster}} = \sum_{c=0}^{19} p_c \cdot \boldsymbol{E}_c \in \mathbb{R}^{32}$$

$p_c$: GMM 사후 확률, 구현: `cluster_probs @ embedding.weight` ($[B, 20] \times [20, 32]$)

hard assignment와 달리 경계 고객의 예측이 클러스터 할당 변동에 민감하지 않다.

# 5. 학습 전략

## 5.1 2-Phase Training

### Phase 1: Shared Expert Pretrain

- **기간**: `shared_expert_epochs` (기본 15)
- **학습 대상**: 전체 모델 — Shared Experts, CGC, Task Experts, Task Towers
- **adaTT**: 활성 — gradient 추출 및 transfer loss 적용

### Phase 2: Cluster Finetune

- **기간**: `cluster_finetune_epochs` (기본 8)
- **학습 대상**: 클러스터별 Task Expert 서브헤드만
- **adaTT**: **비활성** — Shared Expert가 frozen이므로 gradient 추출 무의미
- **CGC**: frozen — 입력(Expert 출력)이 변하지 않으므로 gating 학습은 과적합 유발

Phase 전환 시 리셋되는 항목:

| 리셋 항목 | 이유 |
|---|---|
| Optimizer | Shared Expert frozen → stale momentum 방지 |
| Scheduler | Phase 2 전용 warmup (2 epoch, Phase 1의 5 epoch보다 짧음) |
| GradScaler | AMP 스케일러 상태 초기화 (loss 스케일 변화) |
| Early stopping | best_val_loss, patience_counter 초기화 |
| CGC Attention | Shared Expert frozen → CGC도 함께 freeze |

adaTT는 Phase 2 종료 후 반드시 복원된다 ( `finally` 블록으로 예외 시에도 보장).

## 5.2 Uncertainty Weighting

**Kendall et al., CVPR 2018** 의 homoscedastic uncertainty 기반 자동 태스크 가중치:

$$\mathcal{L}_i^{\text{weighted}} = \frac{1}{2\sigma_i^2} \cdot \mathcal{L}_i + \frac{1}{2} \log \sigma_i^2$$

$\sigma_i^2 = \exp(\log_\text{var}_i)$: 태스크 $i$의 학습 가능한 불확실성
`log_var` clamp: $[-4.0, 4.0]$, precision clamp: $[0.001, 100.0]$

불확실성이 높은 태스크($\sigma_i^2$ 큼)의 가중치($\frac{1}{2\sigma_i^2}$)가 자동으로 낮아지고, $\log \sigma_i^2$ 정규화 항이 불확실성을 무한히 키우는 것을 방지한다. 18개 태스크의 가중치를 수동 튜닝하는 조합 폭발을 **자동 균형**으로 대체한다.

Uncertainty Weighting은 adaTT **이전에** 적용된다. adaTT의 `task_losses` 입력에는 이미 uncertainty weighting이 반영된 값이 들어온다.

## 5.3 Focal Loss

Lin et al., ICCV 2017 의 class-imbalanced binary classification 손실:

$$\mathrm{FL}(p_t) = -\alpha_t \cdot (1 - p_t)^{\gamma} \cdot \log(p_t)$$

$(1 - p_t)^{\gamma}$ 항이 핵심 — 쉬운 예제($p_t \approx 1$)의 손실을 급격히 감쇠시키고, 어려운 예제($p_t \approx 0$)에 학습을 집중한다.

| 태스크 | weight | Focal $\alpha/\gamma$ | 비고 |
|---|---|---|---|
| has_nba | 1.0 | $\gamma$=2, $\alpha$=0.25 | 표준 이진 분류 |
| will_acquire_* | 1.5 | $\gamma$=2, $\alpha$=0.20 | 양성 비율 극소 → weight 상향 |
| churn_signal | 1.2 | $\gamma$=2, $\alpha$=0.60 | FN 비용 높음 → alpha 상향 |
| nba_primary | 2.0 | CE (multiclass) | 비즈니스 핵심 |
| engagement_score | 1.5 | huber ($\delta$=1.0) | regression, outlier robust |
| next_mcc | 2.0 | CE (multiclass) | 소비처 예측 |

## 5.4 AMP (Automatic Mixed Precision)

Forward pass는 `torch.amp.autocast` 로 fp16 실행. TF32 + cuDNN benchmark로 추가 10 15% 속도 확보:

```
torch.backends.cuda.matmul.allow_tf32 = True
torch.backends.cudnn.allow_tf32 = True
torch.backends.cudnn.benchmark = True
```

LR scheduler: Linear Warmup (5 epoch, start_factor=0.1) → CosineAnnealingWarmRestarts ($T_0$=10, $T_{\mathrm{mult}}$=2). Phase 2에서는 scheduler 리셋 (warmup 2 epoch).

# 6. 구현 참고사항

## 6.1 FP16 주의점

### Focal Loss float32 캐스팅

fp16에서 `focal_weight * bce` 의 중간 결과가 subnormal 범위에 들어가면 NaN 발생 가능. 전체 focal loss 계산을 float32로 수행한다:

```
p_f = pred.squeeze().float().clamp(1e-7, 1 - 1e-7)
```

### adaTT Gradient 추출과 autocast

adaTT의 gradient 추출은 autocast 내에서 이루어지지만, loss 계산 자체는 float32로 캐스팅하여 수치 안정성을 확보한다.

## 6.2 GradScaler Guard

`retain_graph=True` 는 아키텍처상 제거 불가 — `_extract_task_gradients` 는 loss 계산 후 `backward()` 전에 호출되며, 동일 computation graph를 Trainer의 `loss.backward()` 에서 재사용해야 한다.

메모리 영향:

| 요소 | 메모리 | 비고 |
|---|---|---|
| Forward pass graph | 1x | 기준 |
| retain_graph 추가 | ~1x | graph 미해제로 추가 메모리 |
| 18 task gradients | ~0.3x | 각 gradient는 shared_param_size |
| **합계** | **~2.3x** | RTX 4070 12GB에서 batch 16384 가능 |

### Gradient 추출 빈도 최적화

`adatt_grad_interval=10` : 매 step 대신 10 step마다 gradient 추출. EMA 평활화 덕분에 충분히 안정적이며, 계산 오버헤드가 1/10로 감소.

### torch.compiler.disable

`_extract_task_gradients` 에 `@torch.compiler.disable` 데코레이터 적용. `torch.autograd.grad` 는 컴파일된 그래프 내에서 `requires_grad` 추적이 불완전하다. 현재 `torch.compile` 자체가 비활성화(15-태스크 MTL + retain_graph + dynamic shape로 커널 컴파일 수백 개, 첫 epoch 30분+)이지만 방어적으로 적용.

## 6.3 Phase 2 Frozen Layer

Phase 2에서 freeze되는 레이어:

| 레이어 | Frozen | 이유 |
|---|---|---|
| Shared Experts | Yes | Phase 1에서 충분히 학습된 공유 표현 보존 |
| CGC Attention | Yes | Expert 출력 고정 → gating 학습 불필요 |
| adaTT | 비활성 | Shared Expert frozen → gradient 0, 코사인 유사도 무의미 |
| GroupEncoder | No | 클러스터별 특화 학습 (Phase 2 핵심) |
| Task Towers | No | 최종 예측 layer 미세조정 |
| ClusterEmbedding | No | 클러스터 표현 정제 |

**CGC-adaTT 동기화**

adaTT `freeze_epoch` 에서 CGC Attention도 함께 frozen한다. CGC가 계속 학습하면 Expert 가중치가 변경되어 adaTT가 측정한 친화도 관계가 무효화될 수 있다.

```
if freeze_epoch is not None and epoch >= freeze_epoch:
    for param in self.task_expert_attention.parameters():
        param.requires_grad = False
    self._cgc_frozen.fill_(True)
```

`_cgc_frozen` 은 `register_buffer` 로 등록되어 체크포인트 저장/복원 시 상태 유지.

## 6.4 Loss 계산 파이프라인 전체 순서

1. **태스크별 loss 유형 결정** (focal, huber, MSE, NLL, contrastive)
2. **Focal Loss alpha/gamma 적용** (태스크별 양성 클래스 가중치)
3. **Loss weight 적용** (Uncertainty Weighting 또는 고정 가중치)
4. **Evidential loss 가산** (불확실성 추정 보조 손실)
5. **adaTT transfer loss** (gradient 기반 전이 손실 추가)
6. **CGC entropy regularization** (Expert Collapse 방지)

## 6.5 디버깅 가이드

| 증상 | 원인 | 해결 |
|---|---|---|
| NaN loss | fp16 focal loss underflow | float32 캐스팅 확인 |
| 학습 초기 loss 발산 | transfer loss가 원본 지배 | `max_transfer_ratio` 확인 (0.5) |
| Phase 2 RuntimeError | adaTT 비활성화 누락 | `model.adatt = None` 확인 |
| `ValueError` 초기화 | `freeze_epoch <= warmup_epochs` | config 검증 |
| 학습 hang (무응답) | 매 step gradient 추출 | `adatt_grad_interval` 확인 (기본 10) |
| 체크포인트 불일치 | `fill_()` 미사용 | buffer 업데이트 시 in-place 연산 확인 |

건강한 친화도 행렬의 특성:

- 같은 그룹 내: 양의 친화도 ($> 0.3$)
- 다른 그룹 간: 약한 양 또는 중립 ($-0.1 \sim 0.3$)
- 대각선: 1.0
- 전체가 $\pm 1$로 포화되지 않을 것 (포화 시 EMA 감쇠율 조정)

# References


1. Caruana, R. (1997). Multitask Learning. Machine Learning, 28(1).
2. Ma, J. et al. (2018). Modeling Task Relationships in Multi-Task Learning with Multi-Gate Mixture-of-Experts. KDD.
3. Tang, H. et al. (2020). Progressive Layered Extraction (PLE): A Novel MTL Model for Personalized Recommendations. RecSys.
4. Kendall, A., Gal, Y. & Cipolla, R. (2018). Multi-Task Learning Using Uncertainty to Weigh Losses. CVPR.
5. Lin, T.-Y. et al. (2017). Focal Loss for Dense Object Detection. ICCV.
6. Yu, T. et al. (2020). Gradient Surgery for Multi-Task Learning (PCGrad). NeurIPS.
7. Chen, Z. et al. (2018). GradNorm: Gradient Normalization for Adaptive Loss Balancing. ICML.
8. Fifty, C. et al. (2021). Efficiently Identifying Task Groupings for Multi-Task Learning. NeurIPS.
9. Navon, A. et al. (2022). Multi-Task Learning as a Bargaining Game (Nash-MTL). ICML.
10. Liu, B. et al. (2021). Conflict-Averse Gradient Descent for Multi-Task Learning (CAGrad). NeurIPS.
11. Ha, D. et al. (2017). HyperNetworks. ICLR.
12. Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
13. Jacobs, R. et al. (1991). Adaptive Mixtures of Local Experts. Neural Computation.
14. Fedus, W. et al. (2022). Switch Transformers: Scaling to Trillion Parameter Models. JMLR.